CSE 4137

Machine Learning
Module-05
Support Vector Machine



Md. Shahid Uz Zaman
Adjunct Faculty
Dept of CSE, BAUET
szaman22.ruet@gmail.com

Support Vector Machine(SVM)

Week-09

- Introduction
- Implementation
- Python Example

SVM Basics

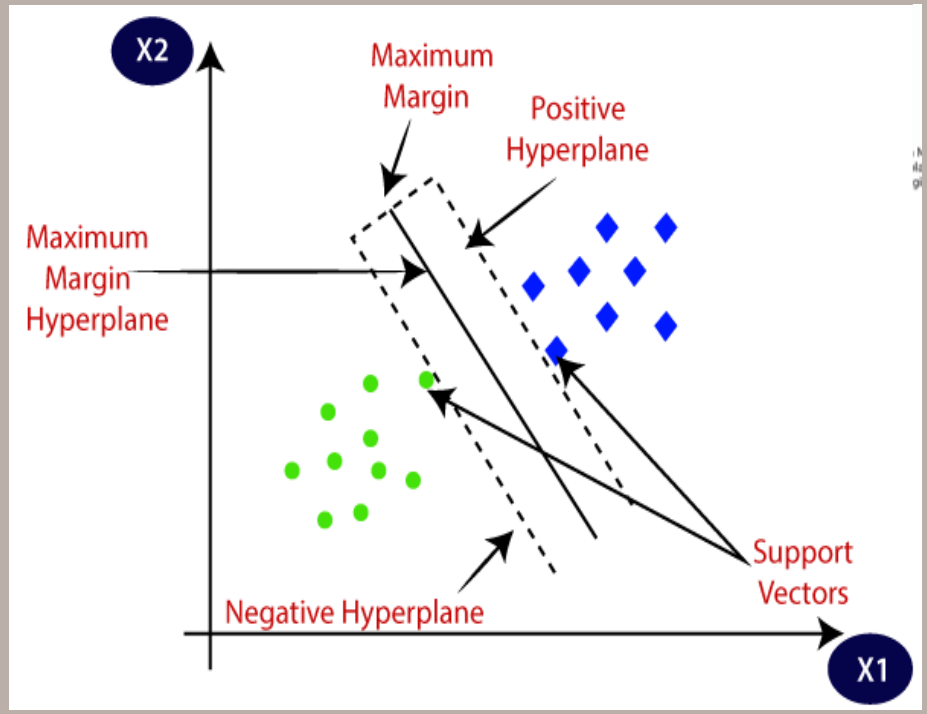
Week-09, c-3

What is SVM?

Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Types of SVM

Linear SVM: Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.

Non-linear SVM: Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

SVM Basics

Week-09, c-3

Hyperplane in SVM

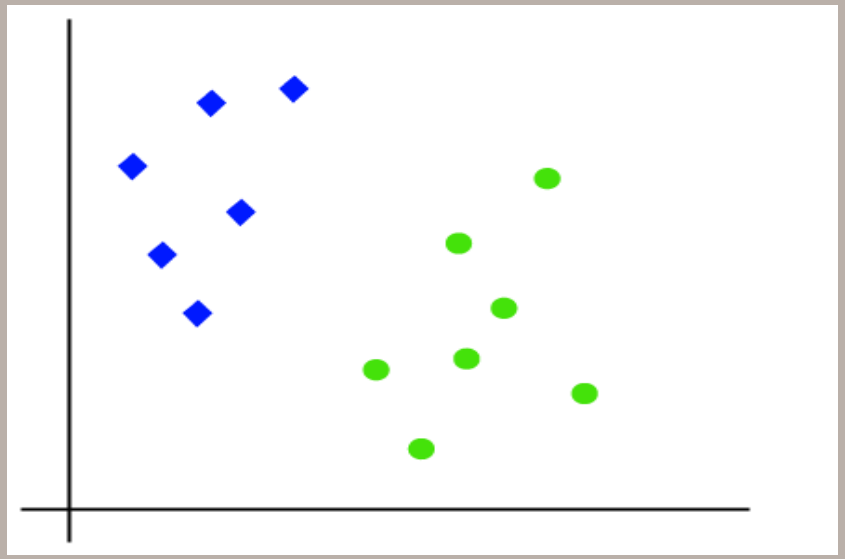
Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.
The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.
We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.
The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a **Support vector**.

How does SVM works?

The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x1 and x2. We want a classifier that can classify the pair(x1, x2) of coordinates in either green or blue. Consider the below image:

Linear SVM:

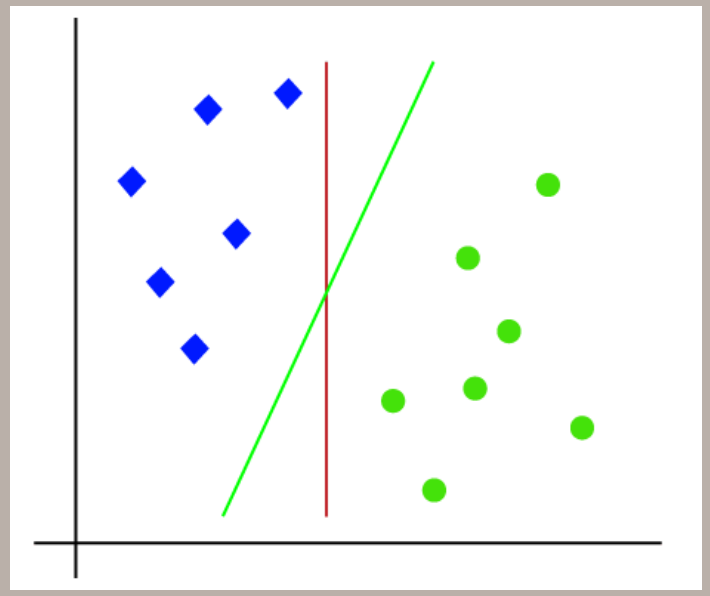
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x1 and x2. We want a classifier that can classify the pair(x1, x2) of coordinates in either green or blue. Consider the below image:

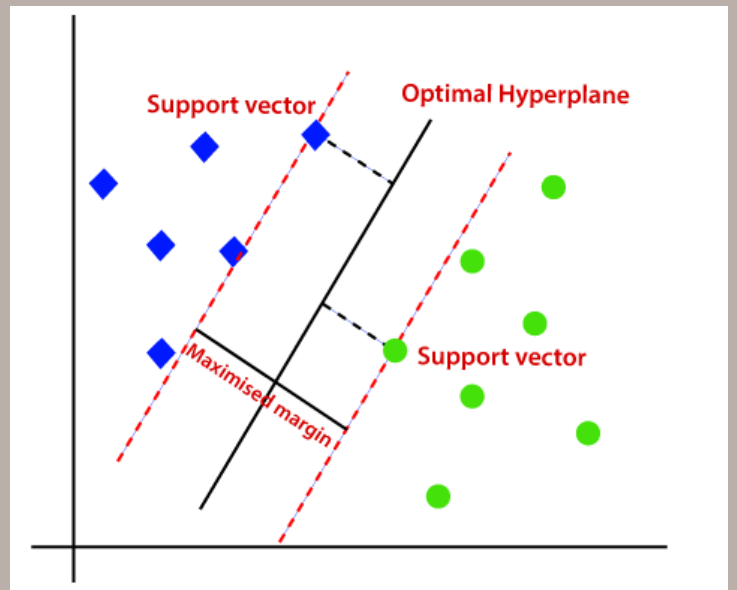


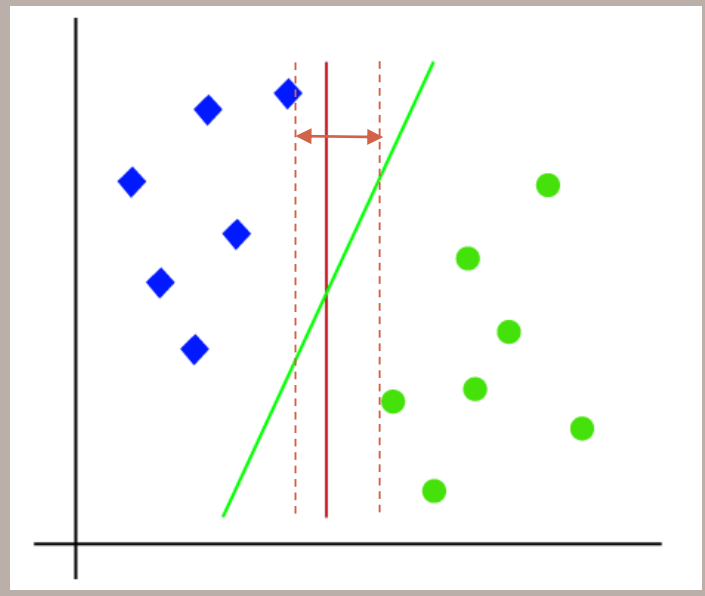
So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

SVM Basics

Week-09, c-3







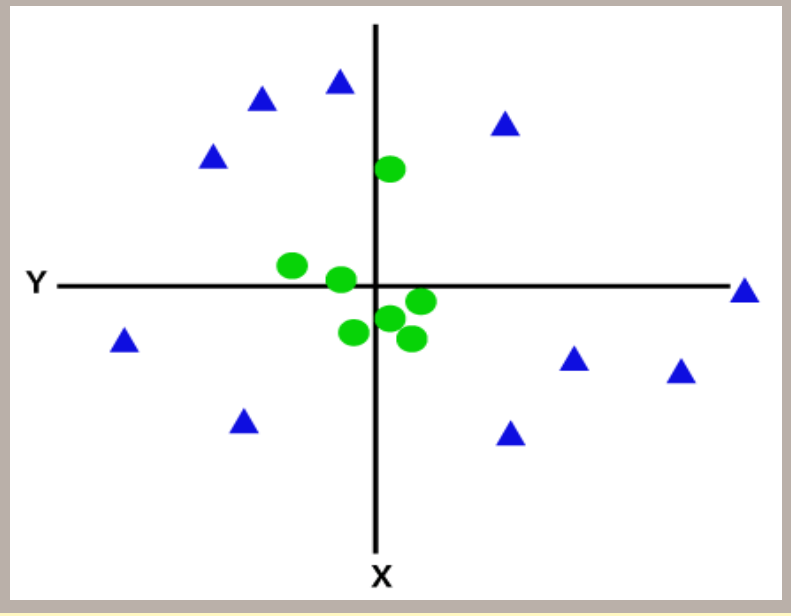
Here in the 2nd figure first a decision boundary is drawn that separates classes and then draws two parallel lines closes to the points in each direction.
Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a hyperplane. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as margin. And the goal of SVM is to maximize this margin. The hyperplane with maximum margin is called the optimal hyperplane.
The hyperplane in first figures has maximum margin than the 2nd one. So 1st hyperplane is selectedc

SVM Basics

Week-09, c-3

Non-Linear SVM:

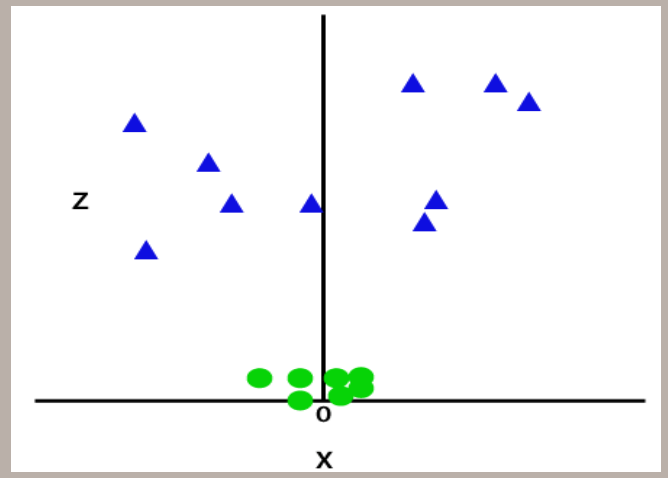
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



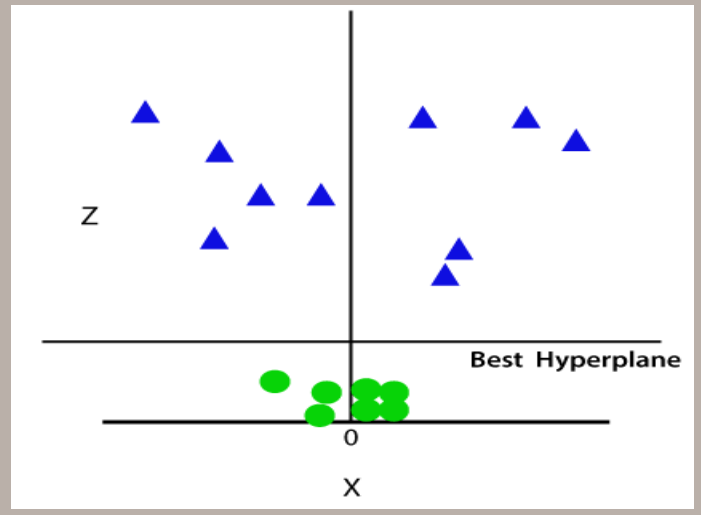
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y, so for non-linear data, we will add a third dimension z. It can be calculated as:

$$z=x^2+y^2$$

By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following

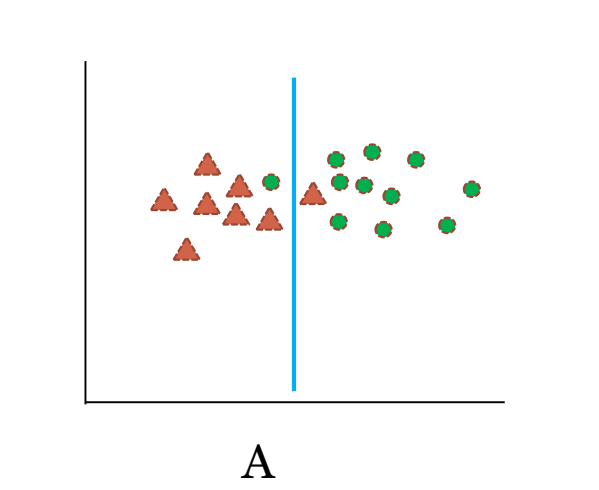


SVM Hyperpameter

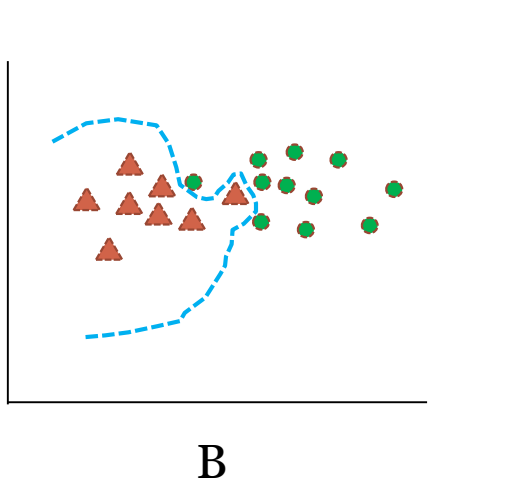
Week-09, c-3

Error Control in SVM

The C parameter in SVM allows us to control the trade-off between the margin and misclassifications. A smaller value of C promotes a wider margin but allows more misclassifications, while a larger value of C prioritizes correct classification at the expense of a narrower margin.



A

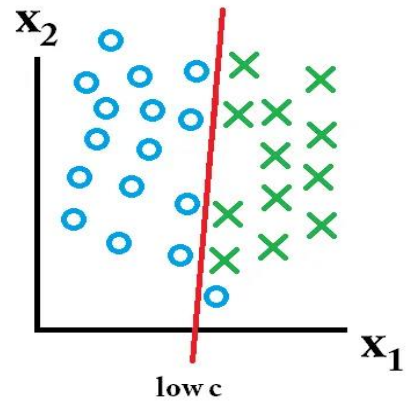


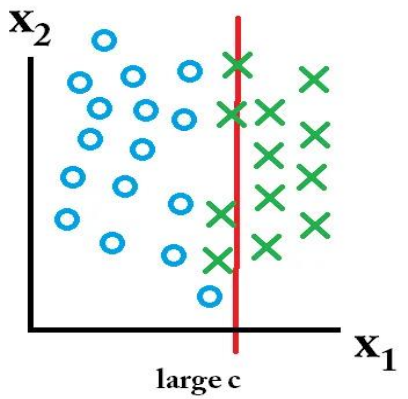
B

In the above image if we choose A then we are getting two errors but if we choose B we do not get any error. But do you think B is the right choice to make a decision boundary? off course NO. A looks better decision boundary even if we are having the errors. In this image, you noticed that error play an important role in SVM.

C in SVM

C- It is a hypermeter in SVM to control error. What does that mean to control error or margin? Let's understand with visualization.





You can see if we have low C means low error and if we have large C means large error.
In low C we have only one error but in case of large C, we have four errors.

So what we have learnt so far that low C less error and large C high error. Note that it does not mean that low error means we have a good model. It totally depends upon on datasets that how much error dataset consists. There is no thumb of rules that low C will work always or high C or medium C.

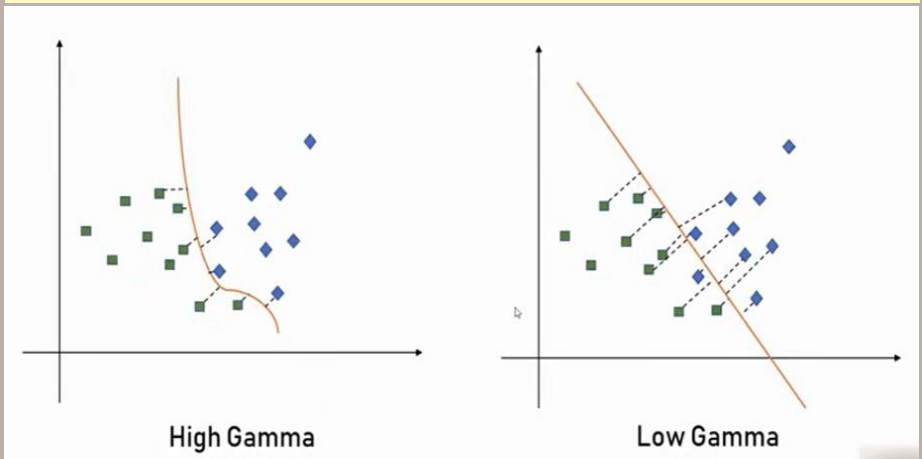
SVM Hyperpameter

Week-09, c-3

Gamma in SVM

Gamma is a hyperparameter which we have to set before training model. Gamma decides that how much curvature we want in a decision boundary.

Gamma high means more curvature.
Gamma low means less curvature.



8

```
import numpy as npimport pandas as pdfrom sklearn.svm import SVC
from sklearn.model_selection import train_test_splitfrom
sklearn.metrics import confusion_matrix
```

```
df=pd.read_csv('F:\\python\\iris.csv')
```

```
X=df.iloc[:,:-1]
y=df.iloc[:,-1]
```

```
sv=SVC()
sv.fit(X_train,y_train)
sv.score(X_test,y_test)
```

```
y_pred=sv.predict(X_test)
confusion_matrix(y_test,y_pred)
```

Week-08

SVM Python Example

SVM: Python Example

Week-09, c-3

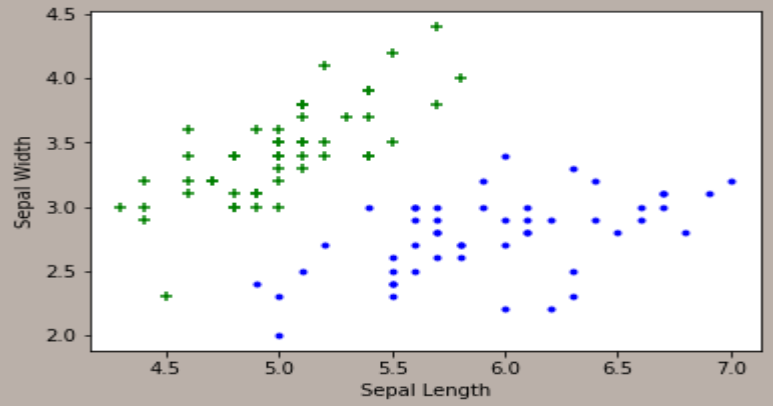
Python Example: Irist Dataset

```
model_linear_kernal = SVC(kernel='poly')
model_linear_kernal.fit(X_train, y_train)
model_linear_kernal.score(X_test, y_test)
```

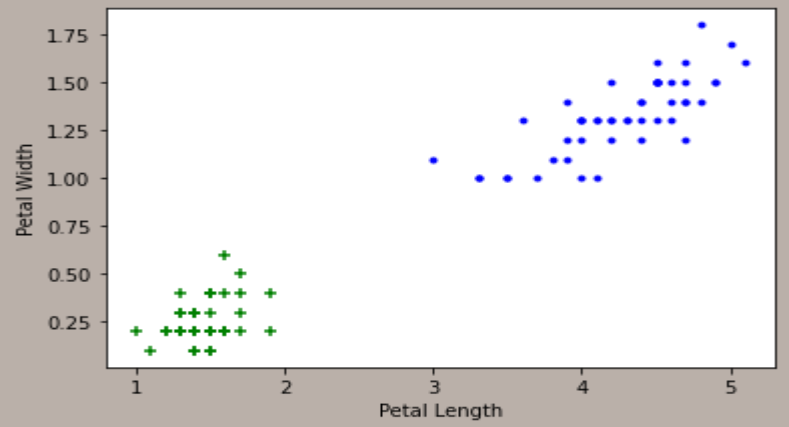
```
df = pd.DataFrame(iris.data,columns=iris.feature_names)
df.head()
```

```
df['target'] = iris.target
df.head()
```

```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.scatter(dfo['sepal length (cm)'], dfo['sepal width (cm)'],color="green",marker='+')
plt.scatter(df1['sepal length (cm)'], df1['sepal width (cm)'],color="blue",marker='.')
```




```
plt.xlabel('Petal Length')
plt.ylabel('Petal Width')
plt.scatter(dfo['petal length (cm)'], dfo['petal width (cm)'],color="green",marker='+')
plt.scatter(df1['petal length (cm)'], df1['petal width (cm)'],color="blue",marker='.')
```




```
from sklearn.model_selection import train_test_split
```

```
X = df.drop(['target'], axis='columns')
y = df.target
```

```
X = df.drop(['target'], axis='columns')
y = df.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

SVM: Python Example

Week-09, c-3

```
from sklearn.svm import SVC
model = SVC()
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

```
model.predict([[4.8,3.0,1.5,0.3]])
```

```
# C hyperparameter
model_C = SVC(C=1)
model_C.fit(X_train, y_train)
model_C.score(X_test, y_test)
```

```
model_C = SVC(C=10)
model_C.fit(X_train, y_train)
model_C.score(X_test, y_test)
```

```
#Gamma hyperparameter
model_g = SVC(gamma=.01)
model_g.fit(X_train, y_train)
model_g.score(X_test, y_test)
```

```
#Gamma hyperparameter
model_g = SVC(gamma=.1)
model_g.fit(X_train, y_train)
model_g.score(X_test, y_test)
```

```
# Kernel (linear/poly/rbf)
model_linear_kernal = SVC(kernel='poly')
model_linear_kernal.fit(X_train, y_train)
model_linear_kernal.score(X_test, y_test)
```

```
model_linear_kernal = SVC(kernel='rbf')
model_linear_kernal.fit(X_train, y_train)
model_linear_kernal.score(X_test, y_test)
```

Week-09, c-3

End of Chapter